



NOWITNESS

Security Audit Report

FluidTokens Smart Contracts Security Audit

Date	April 06, 2026
Client	FluidTokens
Project	Loans V3 - Resellable Loans
Version	0.0.0

Contents

Disclosure	1
Disclaimer	2
Scope	3
Assessment Overview	4
Assessment Components	5
Executive Summary	6
Code Base	9
Severity Classification	10
Finding Severity Ratings	11
Findings	12
[C 501] Continuing Output Value Not Properly Validated	13
[C 502] Duplicate Sale Entries Allow Double Satisfaction	16
[C 503] Token Theft Due to Pool Closure	20
[M 301] DOS Attack via Empty LenderBondsInfo	23
[L 201] Minimum ADA requirement not accounted for	25
[I 101] Bond Sale Design Limitation	27
[I 102] Redundant Accesses to Transaction	30

Disclosure

This document contains proprietary information belonging to No Witness Labs. Duplication, redistribution, or use, in whole or in part, in any form, requires prior written consent from No Witness Labs.

Nonetheless, both the customer **FluidTokens** and No Witness Labs are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the Protocol.

Disclaimer

This review reflects the state of the codebase at the time of assessment and should be regarded as a point-in-time evaluation. Any changes made after the assessment period fall outside the scope of this report.

Although every effort was made to identify potential risks and vulnerabilities, no security review can guarantee the discovery of all issues. The information and recommendations in this report are provided solely for technical purposes and must not be considered investment advice.

To strengthen security assurance, projects are encouraged to seek multiple independent reviews and to supplement audits with practices such as bug bounty programs.

The scope of this audit was limited to the code provided. It did not extend to compiler-level artifacts (e.g., generated UPLC code) or to areas beyond the explicitly reviewed codebase. Additional testing methodologies, including unit testing and property-based testing, were also outside the scope of this engagement.

External oracle dependency

This protocol relies on third-party oracles for price feeds, which were not within the scope of this review. The security of these oracles is critical to the overall safety and functionality of the protocol but was not assessed in this engagement. Users and stakeholders should be aware of this dependency and consider the security implications of relying on external data sources.

Scope

In scope

- On-chain validators and supporting logic at the time of review.
- Datum/redeemer structures and on-chain invariants (value, units/rates, cross-phase checks).
- Use of reference UTxOs for configuration and script parameters.

Out of scope

- Compiler layer / UPLC output and toolchain internals.
- Creation of new unit tests or property-based tests.
- Off-chain libraries, front-end apps, wallet UX (incl. blind-signing), unless explicitly constrained on-chain.
- Third-party protocol internals (bridges/liquidity venues), centralized services, exchanges, oracles, external price feeds.
- Node operations, networking, mempool policy variance, ordering/MEV-like effects.

Assessment Overview

From **March 10, 2026** to **March 19, 2026**, **FluidTokens** engaged No Witness Labs to conduct a comprehensive security assessment of the **Loans V3 - Resellable Loans** protocol. The evaluation was performed in accordance with established industry best practices for code auditing and protocol analysis.

The audit followed a structured five-phase process:

- **Planning** – Defined the assessment scope and objectives, and aligned expectations with the client.
- **Discovery** – Acquired an understanding of the protocol through client discussions, documentation review, and preliminary analysis of architecture and design.
- **Manual Revision** – Conducted an in-depth review of the codebase to identify potential vulnerabilities, design weaknesses, and security risks.
- **Validation** – Verified client-provided fixes and obtained acknowledgments for resolved issues.
- **Reporting** – Documented all confirmed findings, assigned severity levels, and provided actionable recommendations for remediation.

The team also evaluated protocol efficiency and potential optimizations. Findings were prioritized by severity level.

Assessment Components

Manual revision

Our manual code auditing is focused on a wide range of attack vectors, including but not limited to:

- UTXO Value Size Spam (Token Dust Attack)
- Large Datum or Unbounded Protocol Datum
- EUTXO Concurrency DoS
- Unauthorized Data modification
- Multisig PK Attack
- Infinite Mint
- Incorrect Parameterized Scripts
- Other Redeemer
- Other Token Name
- Arbitrary UTXO Datum
- Unbounded protocol value
- Foreign UTXO tokens
- Double or Multiple satisfaction
- Locked Ada
- Locked non Ada values
- Missing UTXO authentication
- UTXO contention

Executive Summary

Protocol Overview

FluidTokens Lending V3 is a Cardano lending protocol implemented as a set of official smart contracts. The logic is intended to be highly customizable, such that the same contracts may be used for different flavours of lending.

At a high level, the protocol supports:

- Static and dynamic liquidity pools (dynamic pools establish borrowable amounts via LTV and oracles), with native multisig wallet support.
- Borrowing from multiple pools in the same transaction, and borrower-created loan requests that lenders can accept.
- Pools that can accept different collaterals, with each loan using exactly one collateral.
- Installments, an initial grace period, and penalty fees for late repayments.
- Four liquidation types (including no-liquidation, dutch auction, and oracle-based variants) and three repayment formulas (including amortization and perpetual loans).
- Unique NFTs identifying pools, requests, loans, and repayments, and bonds (NFTs) representing lender and borrower positions.
- Oracle feeds expressed as tokenX/lovelace with aggregated, pooled, and dedicated feed types, plus CIP-113 (smart tokens) support for principal, collateral, and oracle feeds.
- Optional automatic liquidations and optional automatic principal compounding.

The protocol is described as non-custodial (assets remain owned by users), with repayments sent to the lender-specified address with a specified datum, and is intended to be composable with other Cardano DeFi protocols.

Selling lender positions to a pool

This action allows a pool to spend principal liquidity to purchase lender-bond positions backed by loans that are compatible with the pool. For each processed pool input, the validator checks the referenced lender bonds and loans, routes the lender bond to the pool's configured lender-bond destination, and ensures the pool does not spend more principal than the sum of the referenced loans' current worth.

Lender bond and routing requirements

- Each processed action must include a non-empty `LenderBondsInfo` list.
- Each referenced lender-bond input must contain exactly one lender bond token.
- The referenced loan input must contain the matching loan NFT with the same asset name as the lender bond.
- Duplicate lender bonds are rejected.
- The recreated lender-bond output must:
 - be sent to `poolDatum.lenderBondAddress`,
 - use an inline datum whose hash matches `poolDatum.lenderBondInlineDatumHash`,
 - still contain the lender bond token, and
 - remain bounded in size.

Loan compatibility requirements

- Permissioned pools cannot use this action.
- The loan must match the pool's configured parameters, including principal asset, principal oracle asset, installment structure, installment period, grace period, liquidation mode, repayment mode, repayment window, late repayment penalty, and repayment receipts.
- The loan collateral must equal one of the pool's supported collateral options.
- The loan collateral amount must be at least the collateral required by the pool.
- If `dynamicCollateralPrice` is enabled, the required collateral is computed using collateral and principal oracle feeds; otherwise it is computed without oracles.

- For perpetual loans, the validator compares the pool rate against the loan's current APY; for other loans, it compares against the loan's stored interest rate.

Pool spending and output guarantees

- The validator sums `loanDatum.principalAmount` for all referenced lender bonds.
- The pool input must contain the pool NFT identified by `redeemerAction.poolId`.
- The principal paid by the pool is computed as the reduction in the pool's principal-asset balance between input and output.
- The pool cannot pay more principal than the sum of the referenced loans' current principal amounts.
- The pool output must return to the same address, preserve the same datum, and preserve the input value except for the principal amount spent.
- The transaction validity range must be within one hour.

Code Base

Repository

<https://github.com/FluidTokens/ft-cardano-loans-v3.git>

Branch

resellableLoans

Audited Commit

17b8ba8ff15472cdc12840bc28c1674b4747e0d0

Fixes Verified Commit

803628e1671b2f6eca9e58a86ab647ab02836ac4

Files Audited

Files	SHA256 Checksum
validators/pool/pool_sell_lender_position.ak	2b8c2bc288799a7f8339db4bb8c985d844fc54506e20900948aa187e9d6d164e

Severity Classification

- **Critical:** Vulnerabilities that enable direct theft of assets, permanent loss of funds, or catastrophic protocol failure. These represent immediate and severe financial risk with straightforward exploitation paths.
- **High:** Vulnerabilities that can result in financial loss or significant harm to users or the protocol, though impact may be limited to specific functions or require certain conditions. Exploitation paths exist but may be more complex than Critical Vulnerabilities.
- **Medium:** Vulnerabilities that do not directly result in financial loss but may temporarily impair functionality or create exploitable conditions. These are typically limited to state manipulations and include issues such as front-running susceptibility or logic flaws that compromise transaction integrity.
- **Low:** Low-impact Vulnerabilities with limited attack vectors, minimal attacker incentive, or requiring highly specific scenarios. These typically do not result in financial loss or significant harm to users or the protocol.
- **Informational:** Findings without immediate security or financial implications. These include optimization opportunities, code quality improvements, inconsistencies in naming conventions, design suggestions, and gaps in documentation.

Impact–Likelihood Severity Matrix

	High likelihood	Medium likelihood	Low likelihood
High impact	C	H / M	M
Medium impact	H	M	M / L
Low impact	M	M / L	L

Finding Severity Ratings

The following table defines levels of severity and score range that are used throughout the document to assess vulnerability and risk impact

	Level	Severity	Count
	5	Critical	3
	4	High	0
	3	Medium	1
	2	Low	1
	1	Informational	2
Total Findings			7

Findings

ID	Title	Severity	Status
501	Continuing Output Value Not Properly Validated	Critical	Resolved
502	Duplicate Sale Entries Allow Double Satisfaction	Critical	Resolved
503	Token Theft Due to Pool Closure	Critical	Resolved
301	DOS Attack via Empty <code>LenderBondsInfo</code>	Medium	Resolved
201	Minimum ADA requirement not accounted for	Low	Resolved
101	Bond Sale Design Limitation	Informational	Resolved
102	Redundant Accesses to <code>Transaction</code>	Informational	Resolved

[C 501] Continuing Output Value Not Properly Validated

	Level	Severity	Status
	5	Critical	Resolved

Category: Continuing Output / Optimization

Affected File: `validators/pool/pool_sell_lender_position.ak`

Description

In the `validate_eventual_output_to_pool` function, there are two main issues related to the checks on the output value:

1. A Vulnerability: The function does not check for the preservation of all tokens in the output value. It only checks for the principal asset, the pool NFT, and the length of the output value. If the pool input UTxO contains other tokens, those tokens can be stolen by an attacker (the bond seller) since they are not being validated. This can lead to potential theft of tokens.
2. Inefficiency: The function has three separate checks for the output value, which is inefficient. A more efficient approach would be to calculate the expected output value by deducting the sold principal amount from the pool input UTxO value, and then compare the expected output value with the actual output value. This way, we can ensure that all tokens in the output value are correct and prevent any potential theft of tokens while also improving efficiency.

Relevant code snippet:

```

1  fn validate_eventual_output_to_pool(
2      output: Output,
3      datum: PoolDatum,
4      remainingPrincipalAmountAfterSelling: Int,
5      poolPolicyId: ByteArray,
6      poolId: ByteArray,
7      poolInputOutputAddress: Address,
8      poolInputAssetsLength: Int,
9  ) {
10     expect InlineDatum(outputDatum) = output.datum
11     let containsCorrectAmount =
12         quantity_of(
13             output.value,
14             datum.commonData.principalAsset.policyId,
15             datum.commonData.principalAsset.assetName,
16         ) == remainingPrincipalAmountAfterSelling
17     let containsPoolNFT = quantity_of(output.value, poolPolicyId, poolId) == 1
18     let isDatumCorrect = builtin.equals_data(outputDatum, datum)

```

```

19 let dosProtection = length(flatten(output.value)) == poolInputAssetsLength
20 let correctDestination = poolInputOutputAddress == output.address
21 and {
22     correctDestination,
23     isDatumCorrect,
24     containsPoolNFT,
25     containsCorrectAmount,
26     dosProtection,
27 }
28 }

```

The 3 separate checks for the output value are `containsCorrectAmount`, `containsPoolNFT`, and `dosProtection`. The function does not check for the preservation of all tokens in the output value, which can lead to potential theft of tokens. Additionally, accessing the output value multiple times for different checks is inefficient. It would be more efficient to calculate the expected output value and compare it with the actual output value in a single check, which would also ensure that all tokens in the output value are correct and prevent any potential theft of tokens.

Attack Scenario

An attacker (the bond seller) could exploit the vulnerability by replacing possibly valuable tokens in the output value with less valuable ones (or even with zero-value tokens or spam tokens). Since the function does not check for the preservation of all tokens, the attacker could effectively steal tokens from the pool without being detected.

Recommendation

To address the vulnerability and improve efficiency, it is recommended to optimize the `validate_eventual_output_to_pool` function as follows:

```

1  fn validate_eventual_output_to_pool( Aiken
2      inputValue: Value,
3      output: Output,
4      datum: PoolDatum,
5      poolInputAddress: Address,
6      principalSpent: Int,
7  ) {
8      expect InlineDatum(outputDatum) = output.datum
9      let expectedOutputValue =
10         add(
11             inputValue,
12             datum.commonData.principalAsset.policyId,

```

```

13         datum.commonData.principalAsset.assetName,
14         -principalSpent,
15     )
16     let isDatumCorrect = builtin.equals_data(outputDatum, datum)
17     let correctDestination = poolInputAddress == output.address
18     and {
19         expectedOutputValue == output.value,
20         correctDestination,
21         isDatumCorrect,
22     }
23 }

```

This optimized function calculates the expected output value by deducting the sold principal amount from the pool input UTxO value and then compares it with the actual output value. This ensures that all tokens in the output value are correct and prevents any potential theft of tokens while also improving efficiency.

Benchmark Results

The following benchmark results demonstrate the efficiency of the optimized function compared to the original implementation:

[illegible]

`check_by_inspecting_value` represents the original implementation, while `check_by_constructing_expected_output_value` represents the optimized implementation. The optimized function uses significantly less memory and CPU time, demonstrating its improved efficiency.

Resolution

Resolved in commit hash `b85e902d84d48169354d5babef843abd1d92b288`

[C 502] Duplicate Sale Entries Allow Double Satisfaction

	Level	Severity	Status
	5	Critical	Resolved

Category: Input Validation / Accounting / Double Satisfaction

Affected File: `validators/pool/pool_sell_lender_position.ak`

Description

In the `pool_sell_lender_position_action` validator, we calculate the `principalSpent` by summing up the `principalAmount` of all the `lenderBondInfo` in the `redeemerAction.lenderBondsInfo`. However, there is a critical vulnerability in the current implementation. A Single Lender Bond Can Satisfy Multiple Sale Entries, Causing Pool Overpayment.

Relevant code snippet:

```
1 let principalSpent = Aiken
2   list.foldr(
3     redeemerAction.lenderBondsInfo,
4     0,
5     fn(lenderBondInfo, result) {..
6     },
7   )
```

Although we are checking that each `lenderBondInput` is valid, and a corresponding `outputWithLenderToken` exists and is valid, we are not ensuring that each `lenderBondInput` is only counted once. This means that it is possible for a single `redeemerAction.lenderBondsInfo` to contain multiple entries that reference the same `lenderBondInput`, leading to an inflated `principalSpent` calculation. As a result, the pool could end up overpaying the seller, which is a critical vulnerability that can lead to significant financial losses for the pool.

The validator does not enforce uniqueness across entries in `lenderBondsInfo`. As a result, the same lender bond input, loan reference input, and lender bond output can be referenced multiple times and satisfy multiple sale entries, while `principalSpent` is incremented on each repetition. This allows a single transferred lender position to be counted multiple times during pool accounting.

This vulnerability is not limited to duplicates within a single `redeemerAction.lenderBondsInfo`. Since there is no global uniqueness check across all processed pool inputs, it is also possible for the same lender bond to be counted multiple times across different `poolInput` entries, further exacerbating the potential for overpayment and pool draining.

Attack Scenario

1. A malicious seller creates a `redeemerAction` with multiple `lenderBondInfo` entries that reference the same `lenderBondInputIndex`, `loanRefInputIndex`, and `outputWithLenderTokenIndex`.
2. When the `pool_sell_lender_position_action` validator processes the `redeemerAction`, it calculates the `principalSpent` by summing up the `principalAmount` of all the `lenderBondInfo` entries. Due to the duplicate entries, the `principalSpent` is inflated.
3. The pool overpays the seller based on the inflated `principalSpent`, resulting in significant financial losses for the pool.

Impact

Pool draining: A single transferred lender position can be counted multiple times during pool accounting, allowing a malicious seller to drain excess principal from the pool and leave the pool with less liquidity than warranted by the actual lender bonds acquired.

Recommendation

Introduce a uniqueness check in the `pool_sell_lender_position_action` validator to ensure that each Bond is only counted once in the `principalSpent` calculation.

Enforce that each lender bond asset name may appear at most once across the entire `sellLenderPositionAction`, ensuring that the same bond cannot be counted multiple times across different `poolInput` entries.

This can be achieved by storing each bond's unique identifier `lenderBondAssetName` in a dictionary, and checking for duplicates before adding the `principalAmount` to the `principalSpent`. If a duplicate is detected, the validator should reject the transaction to prevent overpayment.

1. Define a new utility function `indexed_all_with_state` that takes an additional parameter for tracking the state of counted lender bonds.

```
1  /// Variant of `indexed_all` that also threads a state through the
   predicate.
2  /// Processes the list from right to left.
3  /// Returns `True` iff every predicate call returns `True`.
4  /// Does not short-circuit: the predicate is still called for all elements.
5  pub fn indexed_all_with_state(
6      self: List<a>,
7      state: s,
8      predicate: fn(Int, a, s) -> (Bool, s),
9  ) -> Bool {
10     let (finalResult, _finalState) =
11         list.indexed_foldr(
```

 Aiken

```

12     self,
13     (True, state),
14     fn(index, element, (acc, state)) {
15         let (result, newState) = predicate(index, element, state)
16         (result && acc, newState)
17     },
18 )
19 finalResult
20 }

```

2. Replace the use of `indexed_all` in the `pool_sell_lender_position_action` validator with `indexed_all_with_state`, and modify the predicate to include a check for duplicate lender bonds using a dictionary to track counted bonds.

```

1  utils.indexed_all_with_state(
2    poolInputs,
3    dict.empty,
4    fn(index, poolInput, countedLenderBonds) {
5      ...
6      let (principalSpent, newCountedLenderBonds) =
7        list.foldr(
8          redeemerAction.lenderBondsInfo,
9          (0, countedLenderBonds),
10         fn(lenderBondInfo, (result, countedLenderBondsDict)) {
11           ...
12           (
13             result + loanValueInPrincipal,
14             dict.insert_with(
15               countedLenderBondsDict,
16               lenderBondAssetName,
17               True,
18               fn(_, _, _) {
19                 fail @"unexpected duplicate key found in dict."
20               },
21             ),
22           )
23         },
24       )
25       ...
26       and {...

```

```
27     } |> fn(result) { (result, newCountedLenderBonds) }  
28   },  
29 )
```

By implementing this uniqueness check, we can prevent the same lender bond from being counted multiple times, thereby ensuring that the `principalSpent` calculation is accurate and protecting the pool from overpayment vulnerabilities.

Resolution

Resolved in commit hash `b85e902d84d48169354d5babef843abd1d92b288`

[C 503] Token Theft Due to Pool Closure

	Level	Severity	Status
	5	Critical	Resolved

Category: Continuing Output

Affected File: `validators/pool/pool_sell_lender_position.ak`

Description

In the `pool_sell_lender_position_action` validator, there is a condition that allows for the closure of the pool if the remaining principal amount after selling is zero. However, this condition does not account for the presence of other tokens in the pool, which can be taken by an attacker when the pool is closed.

Moreover, if the pool is closed (the pool NFT is burnt) and the lender is expecting auto-compounding, it will fail because the pool no longer exists.

Relevant code snippet:

```

1 //Either the pool must be closed or it must return with the remaining
  liquidity
2 or {
3   and {
4     remainingPrincipalAmountAfterSelling == 0,
5     quantity_of(self.mint, poolPolicyId, redeemerAction.poolId) == -1,
6   },
7   and {
8     remainingPrincipalAmountAfterSelling > 0,
9     validate_eventual_output_to_pool(..),
10  },
11 }
```

 Aiken

Recommendation

To address this critical vulnerability, the option to close the pool should be removed. This means that the condition allowing for the closure of the pool when `remainingPrincipalAmountAfterSelling` is zero should be removed, and only the condition that validates the eventual output to the pool should be retained.

Option 1 (Recommended)

If the recommendations from findings **F201** and **F101** are implemented, the condition should be updated to allow the sale to proceed in case of liquidity shortage in the pool, while still validating the eventual output to the pool.

1. Rename `principalSpent` to `lenderBondsValueInPrincipal` for clarity.

- Update the condition to allow the sale to proceed in case of liquidity shortage in the pool, while still validating the eventual output to the pool.

```

1  let principalAmount =
2    quantity_of(
3      poolInput.output.value,
4      poolDatum.commonData.principalAsset.policyId,
5      poolDatum.commonData.principalAsset.assetName,
6    )
7  let poolOutput = safe_list_at(outputs, index)
8  let principalAmountAfterSelling =
9    quantity_of(
10     poolOutput.value,
11     poolDatum.commonData.principalAsset.policyId,
12     poolDatum.commonData.principalAsset.assetName,
13   )
14  let principalPaid = principalAmount - principalAmountAfterSelling
15  and {
16    ...
17    principalPaid <= lenderBondsValueInPrincipal,
18    validate_eventual_output_to_pool(
19      poolInput.output.value,
20      poolOutput,
21      poolDatum,
22      poolInput.output.address,
23      principalPaid,
24    )
25  }
```

This update ensures that the sale can proceed even if there is a liquidity shortage in the pool, while still validating that the principal paid does not exceed the value of the lender's bonds in terms of the principal asset. Additionally, it removes the option to close the pool, thereby preventing attackers from taking other tokens in the pool when it is closed.

Option 2

If only the recommendation from finding **F201** is implemented, the condition should be updated to remove the option to close the pool, while still validating the eventual output to the pool.

```

1  and {
```

```
2   remainingPrincipalAmountAfterSelling > minAmount,  
3   validate_eventual_output_to_pool(..),  
4 }
```

This update ensures that the pool cannot be closed, and the sale can only proceed if the remaining principal amount after selling is greater than the minimum ADA requirement, while still validating the eventual output to the pool.

Resolution

Resolved in commit hashes:

- `b85e902d84d48169354d5babef843abd1d92b288`
- `cf77b970423587a38c5c0f45147774a0191e7f87`

[M 301] DOS Attack via Empty `lenderBondsInfo`

	Level	Severity	Status
	3	Medium	Resolved

Category: Denial of Service (DoS)

Affected File: `validators/pool/pool_sell_lender_position.ak`

Description

In the `pool_sell_lender_position.ak` validator, there is a potential vulnerability that can lead to a denial of service (DoS) attack. The validator processes a list of `lenderBondsInfo` provided by the redeemer action. If this list is empty, the validator will still succeed and return the consumed pool input back to the pool. This can cause contention problems for other users who want to interact with the pool, as they will have to wait for the attacker's transactions to be processed before they can interact with the pool.

This attack does not cause any loss of funds, but it can lead to a denial of service attack on the pool, as legitimate users may be unable to interact with the pool due to the attacker's actions.

Relevant code snippet from `lib/fluidtokens/types/pool.ak`:

```
1 pub type SellLenderPositionData {
2   lenderBondsInfo: List<SellingLenderBondInfo>,
3   poolId: ByteArray,
4 }
```

Aiken

Relevant code snippet from `pool_sell_lender_position.ak`:

```
1 let principalSpent =
2   list.foldr(
3     redeemerAction.lenderBondsInfo,
4     0,
5     fn(lenderBondInfo, result) {...}
6   )
```

Aiken

Attack Scenario

1. An attacker submits a transaction with an empty `lenderBondsInfo` list to the `pool_sell_lender_position.ak` validator.
2. The validator processes the transaction and succeeds, returning the consumed pool input back to the pool.
3. The attacker continues to submit transactions with empty `lenderBondsInfo`, causing contention problems for other users who want to interact with the pool.
4. Legitimate users may experience delays or be unable to interact with the pool due to the attacker's actions, leading to a denial of service attack on the pool.

Recommendation

To mitigate this vulnerability, the validator should check if the `lenderBondsInfo` list is empty before proceeding with the rest of the validation logic. If it is empty, the validator should reject the transaction. This will prevent attackers from submitting transactions with empty `lenderBondsInfo` and causing contention problems for other users.

By implementing this check, the pool can maintain its availability and ensure that legitimate users can interact with it without being affected by the attacker's actions.

Implementing this check can be done by adding a simple condition before processing the `lenderBondsInfo` list:

```
1 // Add this check before processing the lenderBondsInfo list
2 expect list.is_empty(redeemerAction.lenderBondsInfo) == False
3 // Continue with the rest of the validation logic
4 let principalSpent = ...
```

 Aiken

This condition will ensure that the transaction is rejected if the `lenderBondsInfo` list is empty, thus preventing the DoS attack.

Resolution

Resolved in commit hash `b85e902d84d48169354d5babef843abd1d92b288`

[L 201] Minimum ADA requirement not accounted for

	Level	Severity	Status
	2	Low	Resolved

Category: Value accounting / Minimum ADA requirement

Affected File: `validators/pool/pool_sell_lender_position.ak`

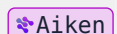
Description

The current implementation of the `pool_sell_lender_position_action` validator does not account for the minimum ADA requirement for the output to the pool.

Relevant code snippet:

```

1  //Either the pool must be closed or it must return with the remaining
   liquidity
2  or {
3    and {
4      remainingPrincipalAmountAfterSelling == 0,
5      quantity_of(self.mint, poolPolicyId, redeemerAction.poolId) == -1,
6    },
7    and {
8      remainingPrincipalAmountAfterSelling > 0,
9      validate_eventual_output_to_pool(..),
10   },
11 }
```

 Aiken

In case the principal is ADA, the output to the pool must be at least the minimum ADA requirement. If the remaining principal amount after selling is less than this minimum, the validator does not allow for the closure of the pool, nor does it allow for the pool to contain more than the remaining principal amount to satisfy the minimum ADA requirement. This means that bond selling cannot happen when it should be allowed to happen.

Impact

This issue can prevent legitimate bond sales from occurring, especially in cases where the remaining principal amount after selling is less than the minimum ADA requirement. This can lead to a lack of liquidity and flexibility for lenders who wish to sell their positions.

Recommendation

To address this issue, `remainingPrincipalAmountAfterSelling` should be compared against a minimum amount that takes into account the minimum ADA requirement. The condition should be updated as follows:

1. Define a `minAmount` variable that represents the minimum ADA requirement if the principal asset is ADA, or 0 otherwise.

```

1 let minAmount =
2   if poolDatum.commonData.principalAsset.policyId == assets.ada_policy_id {
3     2_000_000
4   } else {
5     0
6   }

```

Aiken

2. Update the condition to account for the minimum ADA requirement:

```

1 or {
2   and {
3     remainingPrincipalAmountAfterSelling <= minAmount,
4     remainingPrincipalAmountAfterSelling >= 0,
5     quantity_of(self.mint, poolPolicyId, redeemerAction.poolId) == -1,
6   },
7   and {
8     remainingPrincipalAmountAfterSelling > minAmount,
9     validate_eventual_output_to_pool(..),
10  },
11 }

```

Aiken

This update ensures that in case the principal is ADA, if the remaining principal amount after selling is less than the minimum ADA requirement, the pool can be closed, and the bond can be sold. If the remaining principal amount is greater than the minimum ADA requirement, the validator will continue to validate the eventual output to the pool as before.

Resolution

Resolved in commit hash `cf77b970423587a38c5c0f45147774a0191e7f87`

[I 101] Bond Sale Design Limitation

	Level	Severity	Status
	1	Informational	Resolved

Category: Design

Affected File: `validators/pool/pool_sell_lender_position.ak`

Description

The current implementation of the `pool_sell_lender_position_action` validator requires that bonds be sold for their full value by ensuring that the calculated remaining principal amount after sale is non-negative. This means that bonds cannot be sold for less than their face value. However, in a real-world scenario, it may be desirable for the bondholder to be able to sell their bond for less than its face value when liquidity in the pool is insufficient to buy it at its full value. This would provide more flexibility for lenders who wish to sell their positions, especially in distressed sale scenarios where they may need to sell quickly and may not be able to find a pool with sufficient liquidity to buy the bond at its full value. Allowing sales at a discount in such scenarios could create more opportunities for lenders to sell their positions when needed.

Relevant code snippet:

```

1 let remainingPrincipalAmountAfterSelling = 
2   quantity_of(
3     poolInput.output.value,
4     poolDatum.commonData.principalAsset.policyId,
5     poolDatum.commonData.principalAsset.assetName,
6   ) - principalSpent

1 or { 
2   and {
3     remainingPrincipalAmountAfterSelling == 0,
4     quantity_of(self.mint, poolPolicyId, redeemerAction.poolId) == -1,
5   },
6   and {
7     remainingPrincipalAmountAfterSelling > 0,
8     validate_eventual_output_to_pool(..),
9   },
10 }
```

In the above code snippet, the validator enforces that `remainingPrincipalAmountAfterSelling` must be greater than or equal to 0:

```
remainingPrincipalAmountAfterSelling >= 0
```

Substituting the value of `remainingPrincipalAmountAfterSelling`, we get:

```
pool_input_principal_amount - principalSpent >= 0
```

Which can be rearranged to:

```
pool_input_principal_amount >= principalSpent
```

This means that the `principalSpent` cannot exceed the current principal amount in the pool. This effectively prevents the bond from being sold for less than its worth, which may not always be desirable or necessary in a real-world scenario.

Impact

When liquidity in the pool is insufficient to buy the bonds at their full value, the current implementation of the validator would prevent the bonds from being sold for less than their full value. This can lead to a lack of flexibility for lenders who wish to sell their positions, especially in distressed sale scenarios where they may need to sell quickly and may not be able to find a pool with enough liquidity to buy the bonds for their full worth.

Recommendation

Allow the sale to proceed in case of liquidity shortage in the pool by allowing the `remainingPrincipalAmountAfterSelling` to be negative.

1. If the recommendation from finding **F201** is implemented:

Remove this check: `remainingPrincipalAmountAfterSelling >= 0`. So the condition would be updated as follows:

```
1  or {
2    and {
3      remainingPrincipalAmountAfterSelling <= minAmount,
4      quantity_of(self.mint, poolPolicyId, redeemerAction.poolId) == -1,
5    },
6    and {
7      remainingPrincipalAmountAfterSelling > minAmount,
8      validate_eventual_output_to_pool(..),
9    },
10 }
```

2. If the recommendation from finding **F201** is not implemented:

Replace `remainingPrincipalAmountAfterSelling == 0` with the more flexible `remainingPrincipalAmountAfterSelling <= 0`. So the condition would be updated as follows:

```
1  or {
2    and {
```

```
3      // Allow the bond to be sold for less than its face value when liquidity in the
      pool is insufficient to buy it at its full value
4      remainingPrincipalAmountAfterSelling <= 0,
5      quantity_of(self.mint, poolPolicyId, redeemerAction.poolId) == -1,
6  },
7  and {
8      remainingPrincipalAmountAfterSelling > 0,
9      validate_eventual_output_to_pool(..),
10 },
11 }
```

Resolution

Resolved in commit hash `cf77b970423587a38c5c0f45147774a0191e7f87`

[I 102] Redundant Accesses to Transaction

	Level	Severity	Status
	1	Informational	Resolved

Category: Optimization

Affected File: `validators/pool/pool_sell_lender_position.ak`

Description

The function `is_loan_compatible_with_pool` is being called inside a nested loop, and it is accessing the transaction multiple times at each iteration. This is redundant and can be optimized by passing only the necessary data to the function instead of passing the whole transaction.

Relevant code snippet:

```
1  fn is_loan_compatible_with_pool(Aiken
2      loanDatum: LoanDatum,
3      poolDatum: PoolDatum,
4      loanValueInPrincipal: Int,
5      collateralIndex: Int,
6      loanCollateralAmount: Int,
7      collateralOracleRefInputIndex: Int,
8      principalOracleRefInputIndex: Int,
9      self: Transaction,
10 ) {...}
```

The argument `self` is being used to access the following values:

- `self.validity_range.lower_bound.bound_type`
- `self.validity_range.upper_bound.bound_type`
- `self.reference_inputs`
- `self.redeemers` All of these variables do not change during the iterations of the nested loop, and are fixed throughout the execution of the validator.

The argument `loanValueInPrincipal` is also redundant since it can be accessed from `loanDatum.principalAmount`.

There are also accesses to `self.inputs`, `self.outputs`, and `self.mint` inside the main loop that iterates over `poolInputs`, but those values are not changing during the iterations of the loop, so they can also be extracted before the loop.

These redundant accesses to the transaction can lead to performance issues, especially if the loops have a large number of iterations. By extracting the necessary values from the transaction once before the loops, we can reduce the number of accesses to the transaction and improve the performance of the validator.

Recommendation

To optimize the performance of the function, we should extract the necessary values from the transaction once before the loops and pass them as parameters to the function instead of passing the whole transaction. This will reduce the number of accesses to the transaction and improve the performance of the function.

1. Extract the necessary values from the transaction at the start of the validator:

```
1  let Transaction {
2    inputs,
3    outputs,
4    reference_inputs,
5    redeemers,
6    mint,
7    validity_range,
8    ..
9  } = self
10 expect Finite(validFrom) = validity_range.lower_bound.bound_type
11 expect Finite(validTo) = validity_range.upper_bound.bound_type
```

2. Replace any access to `self.inputs`, `self.outputs`, `self.reference_inputs`, `self.redeemers`, `self.mint`, `validFrom`, and `validTo` with the extracted variables.
3. Modify the function call to pass these extracted values instead of `self`, and remove the redundant `loanValueInPrincipal` argument:

```
1  fn is_loan_compatible_with_pool(
2    loanDatum: LoanDatum,
3    poolDatum: PoolDatum,
4    collateralIndex: Int,
5    loanCollateralAmount: Int,
6    collateralOracleRefInputIndex: Int,
7    principalOracleRefInputIndex: Int,
8    validFrom: Int,
9    validTo: Int,
10   reference_inputs: List<Input>,
11   redeemers: Pairs<ScriptPurpose, Redeemer>,
12 ) {...}
```

By implementing these changes, we can optimize the performance of the validator and reduce the number of redundant accesses to the transaction.

Resolution

Resolved in commit hash `b85e902d84d48169354d5babef843abd1d92b288`

End of Report



No Witness Labs — Confidential

Copyright © 2026